VÍA

Voz, Imagen y Accesibilidad

SEGUNDA ENTREGA

Del prototipo al producto funcional

Diseño, prototipado y evaluación inicial de la aplicación inclusiva. En esta entrega VÍA dejó de ser una maqueta estática: es una aplicación Flutter real, instalable en un teléfono Android, que lee en voz alta, escucha comandos y se repinta al instante según el perfil de accesibilidad elegido.

✓ Prototipo funcional

✓ Evaluación heurística (Nielsen)

✓ Protocolo de prueba con usuarios

Curso: Interacción entre Hombre y Máquina · Universidad Tecnológica del Perú
Integrantes — Enrique Sebastian Poma Nizama · U23200217

RESUMEN EJECUTIVO

Qué cambió desde la primera entrega

La Entrega 1 propuso y diseñó la solución (teoría, requisitos, mockups estáticos). Esta entrega la construye: hay código fuente real, corriendo en un dispositivo real, que se puede tocar, hablar y escuchar.

1

Entrega 1 → Entrega 2

De 6 mockups en HTML/PNG (sin funcionar) a 32 archivos Dart, 3 676 líneas de código, compilando a APK real para Android.

2

Alcance: profundidad, no ancho

La consigna pide cubrir 5 grupos (visual, auditivo, motor, cognitivo, adultos mayores). Decidimos, como parte del DCU, profundizar en uno solo — discapacidad visual — en vez de hacer cinco soluciones superficiales. Se explica el porqué en la página siguiente.

3

Nuevo: personalización de color

Más allá de lo pedido: el usuario cambia el esquema de color en vivo (alto contraste oscuro, claro, o una paleta segura para daltonismo) sin reiniciar la app.

32

archivos Dart, organizados en 4 capas (core, data, features, shared) — la misma arquitectura funcional propuesta en la Entrega 1, ahora hecha código.

3 676

líneas de código fuente, comentadas en español, explicando el porqué de cada decisión no obvia.

6

pantallas completas y navegables: perfil, inicio por voz, lector adaptativo, Ojo IA, ajustes y ayuda humana (simulada).

DISEÑO CENTRADO EN EL USUARIO

Las cinco etapas, aplicadas a VÍA

Siguiendo el proceso de DCU visto en el curso (y las heurísticas de Nielsen que se evalúan en la página 7), cada etapa dejó una evidencia concreta dentro del propio proyecto.

ETAPA	QUÉ SE HIZO	EVIDENCIA
1 · Empatizar / investigar	Revisión de literatura sobre accesibilidad para personas ciegas (VizWiz, VizWiz Grand Challenge, estudio de Adnin & Das sobre IA generativa y usuarios ciegos) y de datos duros (OMS, WebAIM, INEI).	docs/01-propuesta.md
2 · Definir	22 requisitos funcionales y 11 no funcionales, cada uno trazable a una fase del plan de trabajo.	docs/02-requisitos.md
3 · Idear / diseñar	Sistema de diseño con colores verificados matemáticamente (fórmula de contraste WCAG), tipografía y medidas táctiles mínimas de 48×48 dp.	docs/04-design-system.md
4 · Prototipar	Primero en baja fidelidad (HTML estático), después en alta fidelidad **funcional**: una app Flutter real que compila, instala y corre en un teléfono Android.	32 archivos · lib/
5 · Evaluar	Evaluación heurística de Nielsen sobre el prototipo funcional (página 7-8) + protocolo de prueba con usuarios reales y escala SUS, listo para ejecutarse (página 9).	Pendiente de correr con usuarios

La etapa 5 queda documentada como protocolo, no como resultados: una evaluación con personas reales requiere agendarla aparte de esta entrega — ver página 9.

PARA SUSTENTAR EL TRABAJO

Cómo está armado el código

VÍA separa el código en 4 capas, para que cada pieza tenga un único trabajo y se pueda explicar sin enredarse con las demás.

CAPA	QUÉ VIVE AHÍ	EJEMPLO REAL
core/	Tema visual, accesibilidad (voz+vibración+anuncio), comandos de voz	`anunciador.dart`
data/	Modelos y servicios — cada paquete externo (TTS, cámara, OCR) vive envuelto en un único archivo	`tts_service.dart`
features/	Una carpeta por pantalla: la vista dibuja, el controlador decide	`ojo_ia_controller.dart`
shared/	Botones y tarjetas reutilizados en varias pantallas	`tarjeta_accesible.dart`

Regla que no se negocia: ningún widget llama a un paquete externo directamente. Si mañana se cambia el motor de voz, se toca *un solo archivo* — nunca las 14 pantallas.

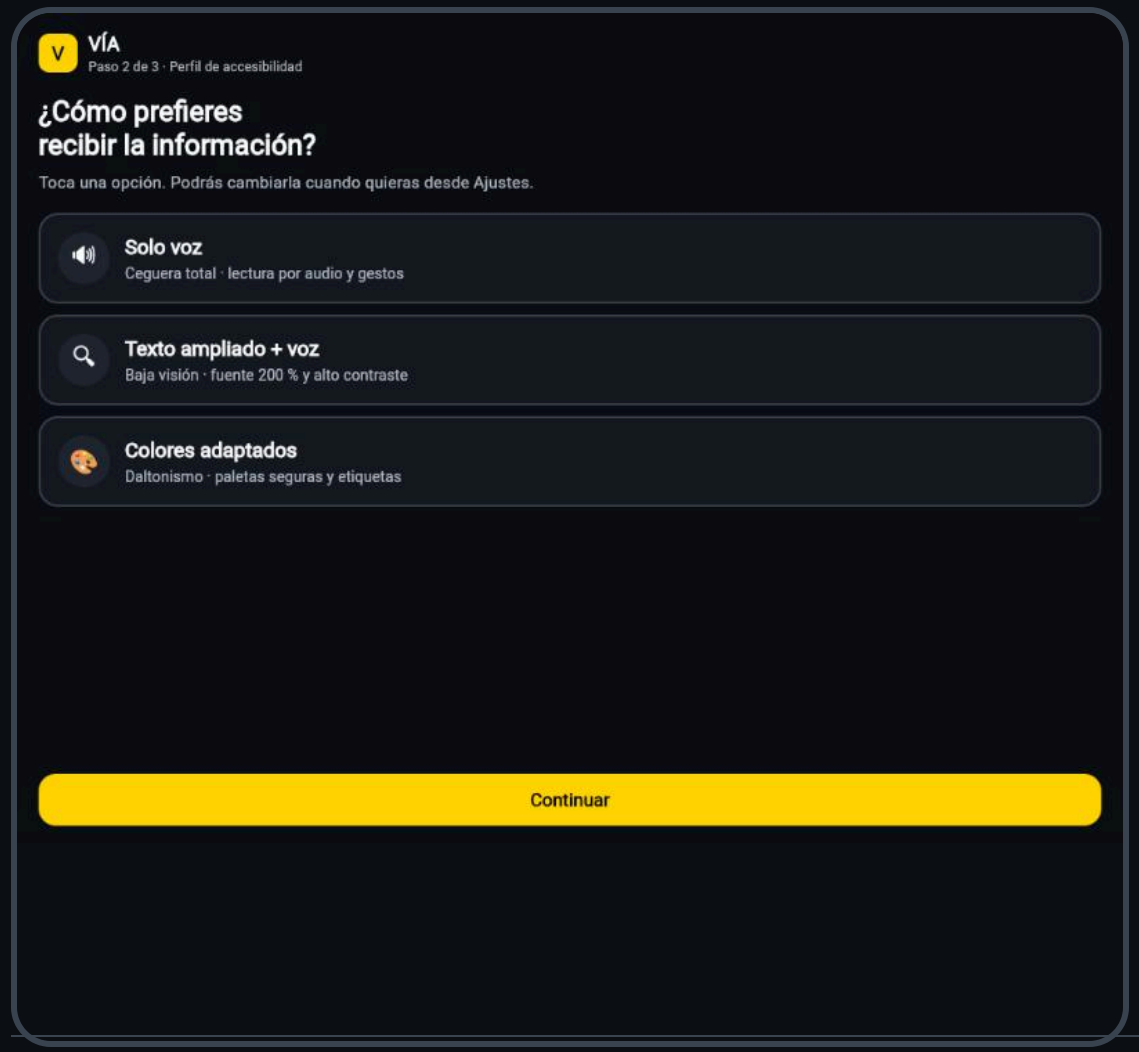
El Anunciador — la pieza más usada

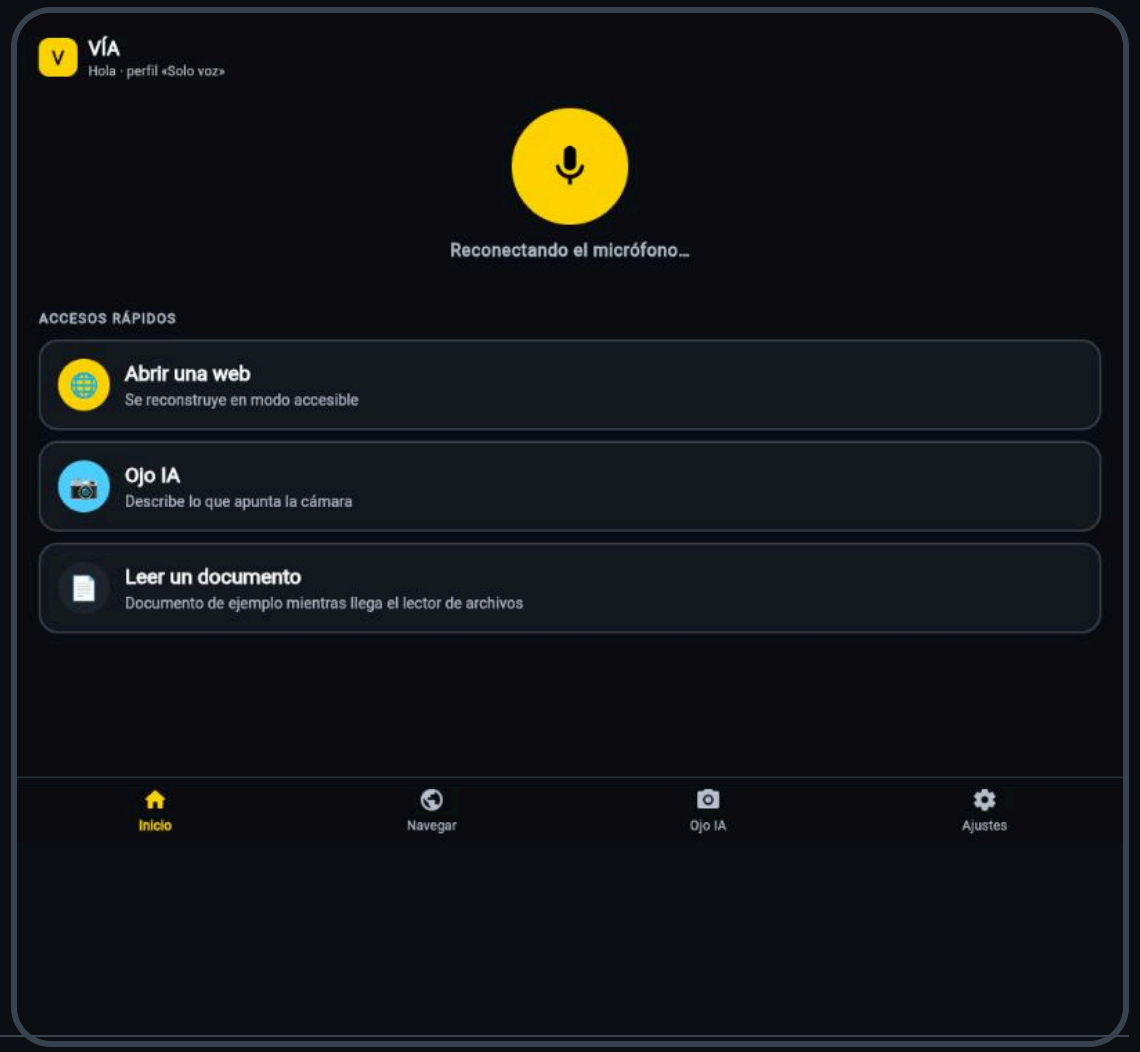
Une voz + vibración + lector de pantalla en una sola llamada. Toda acción del usuario pasa por aquí (regla de accesibilidad: si algo cambia y no se anuncia, para un usuario ciego ese cambio no existe).

```
class Anunciador {
  Future<void> anunciar(
    String mensaje, {
    bool interrumpir = false,
    TipoVibracion vibracion =
        TipoVibracion.cambioFoco,
  }) async {
    vibracion.ejecutar();
    SemanticsService.announce(
        mensaje, TextDirection.ltr);
    await _tts.hablar(mensaje,
        interrumpir: interrumpir);
  }
}
```

PROTOTIPO FUNCIONAL · CAPTURAS REALES

Perfil e inicio por voz





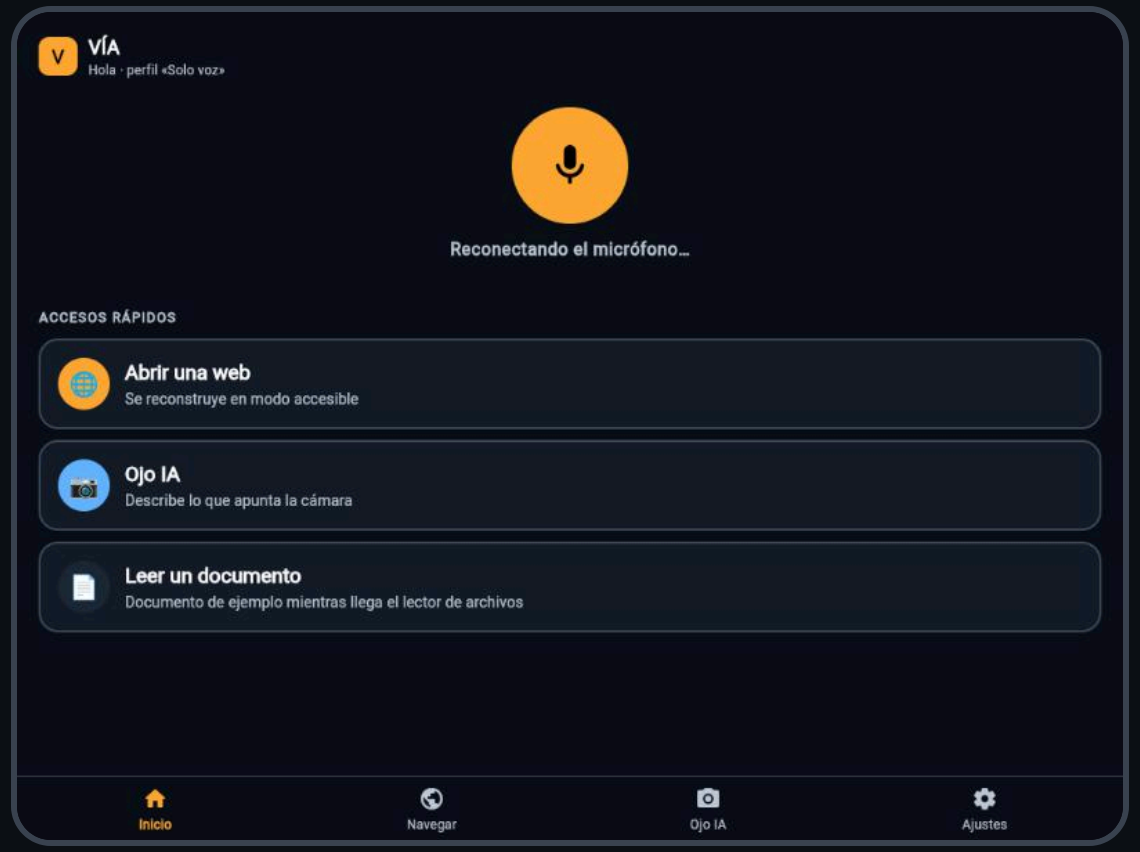
Capturas tomadas de la app real corriendo (no mockups). Izquierda: P-01, perfil de accesibilidad. Derecha: P-02, inicio con escucha activa y accesos rápidos.

LO QUE PIDE EL PROFESOR, Y ALGO MÁS

Personalización de color en vivo

Un cambio en Ajustes repinta la app **entera**, al instante, sin reiniciar nada — incluida la paleta segura para daltonismo (protanopía/deuteranopía), con contraste verificado matemáticamente igual que el esquema por defecto.





Mismo estado de la app, un toque después: el esquema "Colores adaptados (daltonismo)" seleccionado en Ajustes (izquierda) repintó también la pantalla de Inicio (derecha) sin recargar la app.

EVALUACIÓN HEURÍSTICA

Las 10 heurísticas de Nielsen, aplicadas a VÍA

HEURÍSTICA	CÓMO LA CUMPLE VÍA
1. Visibilidad del estado	Toda acción se confirma por voz + vibración + anuncio al lector de pantalla a la vez (clase `Anunciador`). El orbe de Inicio muestra "● ESCUCHANDO…" en tiempo real.
2. Coincidencia con el mundo real	Los mensajes de voz dan primero el dato, después el adorno ("S/ 87.40, total a pagar"), sin jerga técnica ni nombres de íconos.
3. Control y libertad	Comandos "repetir", "parar", "más rápido/lento" en cualquier pantalla; el lector permite anterior/siguiente/pausar en cualquier momento.
4. Consistencia y estándares	Un solo sistema de diseño (`ColoresVia`, `TemaVia`) usado por las 6 pantallas: mismos 48 dp mínimos, mismo radio de tarjeta, mismo patrón de anuncio.
5. Prevención de errores	Ojo IA avisa por voz antes de capturar ("hay poca luz", "acércate") en vez de dejar que el usuario tome una foto inservible.
6. Reconocer antes que recordar	Los "Accesos rápidos" de Inicio y los comandos de voz se anuncian explícitamente ("puedes decir: leer documento, abrir cámara o ajustes") en vez de exigir memorizarlos.
7. Flexibilidad y eficiencia	Cada acción tiene equivalente táctil *y* de voz — un usuario con resto visual toca, uno sin visión habla.
8. Diseño estético y minimalista	Una sola acción principal por pantalla (amarillo); nada compite por atención. Jerarquía tipográfica fija (26/19/18/16 sp).
9. Ayudar a reconocer y resolver errores	"No entendí. Puedes decir: leer documento, abrir cámara o ajustes" en vez de un error seco — nunca un mensaje técnico.
10. Ayuda y documentación	P-06 conecta con un voluntario humano cuando la IA no alcanza (simulado, ver página 9 de la Entrega 1).

EVALUACIÓN HEURÍSTICA · HALLAZGOS

Lo que la evaluación encontró para mejorar

Una evaluación heurística que no encuentra nada que mejorar no es rigurosa. Estos son los tres hallazgos reales de recorrer el prototipo funcional, con su heurística y severidad.

Severidad baja

H10 — Comandos no descubribles de entrada

El onboarding no enseña todavía los comandos de voz disponibles (llegan recién en Inicio). Un usuario nuevo podría no saber qué decir la primera vez.

Mejora propuesta: un comando "ayuda" que liste los comandos disponibles, accesible desde cualquier pantalla.

Severidad baja

H1 — Silencio durante procesos largos

Mientras el OCR procesa una foto o el lector web descarga una página, solo hay un texto estático ("Leyendo…"), sin progreso audible intermedio.

Mejora propuesta: un sonido de progreso cada ~3 s en esperas largas.

Limitación de alcance

H5 — Guía de encuadre simplificada

Ojo IA avisa de la **luz** real del encuadre (medida de verdad), pero no detecta si el documento está "cortado" o "lejos" con precisión — eso pide reconocer bordes, con una librería de visión más pesada (OpenCV), fuera del alcance de esta fase. Documentado en el propio código.

SIGUIENTE PASO DE ESTA ENTREGA

Protocolo de prueba con usuarios (escala SUS)

Listo para ejecutarse con 3 a 5 personas (ciegas, con baja visión, o con los ojos vendados simulando ceguera total, como indica la propia guía de desarrollo del proyecto).

Tareas a cronometrar

1. Elegir un perfil de accesibilidad sin ayuda (P-01).
2. Decir "leer documento" y escuchar el resultado completo.
3. Pausar y reanudar la lectura por voz.
4. Cambiar la velocidad de voz desde Ajustes.
5. Decir "ayuda humana" y llegar hasta P-06.

Cuestionario SUS (10 ítems, 1-5)

1. Usaría esta app frecuentemente · 2. La encontré innecesariamente compleja · 3. La app me pareció fácil de usar · 4. Necesitaría ayuda de alguien técnico · 5. Las funciones están bien integradas · 6. Hay demasiada inconsistencia · 7. La mayoría aprendería a usarla rápido · 8. La sentí incómoda de usar · 9. Me sentí segura(o) usándola · 10. Necesité aprender mucho antes de empezar.

Cómo se calcula el puntaje

Ítems impares: puntaje − 1. Ítems pares: 5 − puntaje. Sumar los 10, multiplicar por 2.5 → escala 0-100. Un puntaje sobre 68 se considera por encima del promedio de usabilidad (Brooke, 1996).

Esta página es la plantilla, no el resultado — se completa al correr la prueba con personas reales, antes de la presentación final de lanzamiento.

CIERRE

Conclusiones y lo que falta

Lo que ya funciona (fases 0-6 del plan)

Entorno, tema visual con 3 esquemas de color, voz (TTS), perfil de accesibilidad, navegación por voz, lector adaptativo con contenido de ejemplo, y ajustes con todo aplicándose en vivo.

Lo que sigue (fases 7-12 del plan)

Cámara + OCR en el dispositivo, historial, lector web completo (ya con servicio implementado), descripción de imágenes con IA en la nube, y la simulación de ayuda humana.

Decisión de alcance, otra vez, para que quede explícito

El profesor pidió cubrir 5 grupos de usuarios; VÍA profundiza en uno. Esto sigue el propio Diseño Centrado en el Usuario: diseñar "para todos" sin investigación específica de cada grupo produce soluciones superficiales que no sirven de verdad a nadie. El marco teórico de la Entrega 1 (Shneiderman, 2000; Story, Mueller & Mace, 1998) respalda exactamente esta decisión: interfaces multicapa y flexibles, no un diseño único forzado a todos los perfiles.